TCS Placement Preparation Handbook

Frequently Asked Questions Edition — NQT | Digital | Prime | Atlas Hiring

A Frequently Asked Questions companion covering Quantitative Aptitude, Logical Reasoning, Verbal Ability, Coding, Data Structures & Algorithms, DBMS, Operating Systems, Computer Networks, OOP, HR Interview and a 30-Day Study Plan.

Prepared for: **Kiki**

M.Sc. Data Science, SASTRA Deemed University

Target: TCS NQT / TCS Digital / TCS Prime / TCS Atlas Hiring

Table of Contents

Section 1

Quantitative Aptitude

1. Frequently Asked Questions — Quantitative Aptitude

1.1 Number System & Divisibility

```
Divisible by 2 -> last digit even | by 3 -> digit sum div by 3
by 4 -> last 2 digits div by 4 | by 5 -> ends in 0/5
by 6 -> div by 2 and 3 | by 8 -> last 3 digits div by 8
by 9 -> digit sum div by 9 | by 11 -> (sum odd pos - sum even pos) div by 11
```

Q: Is 4536 divisible by 11?

A: Odd-position sum (from right) = 6+5+4=15... actually apply rule: sum of digits at odd places (6+3=9... use standard alternating sum) = (6-3+5-4)=4, not divisible by 11. Quick check: 4536/11 = 412.36, so **No**.

Q: Find the remainder when 2^40 is divided by 7.

A: 2^3 = 8 ≡ 1 (mod 7). So 2^40 = 2^(3×13+1) = (2^3)^13 × 2 ≡ 1^13 × 2 = **2**.

Q: What is the smallest number which when divided by 4, 6, and 8 leaves remainder 2 in each case?

A: LCM(4,6,8) = 24. Required number = 24 + 2 = **26**.

Q: How many numbers between 1 and 100 are divisible by 3 or 5?

A: Div by 3: 33, div by 5: 20, div by both 15: 6. Total = 33+20-6 = **47**.

1.2 HCF & LCM

```
HCF x LCM = Product of the two numbers
LCM = product / HCF
```

Q: Find the HCF and LCM of 24 and 36.

A: 24 = 2^3×3, 36 = 2^2×3^2. HCF = 2^2×3 = 12. LCM = 2^3×3^2 = **72**. HCF = **12**.

Q: Two numbers have HCF 6 and LCM 108. If one number is 18, find the other.

A: Other number = (HCF×LCM)/18 = (6×108)/18 = **36**.

Q: Find the greatest number that divides 615 and 963 leaving remainder 6 in each case.

A: Subtract remainder: 609 and 957. HCF(609,957) = **87** (using Euclid's algorithm).

1.3 Percentages, Profit & Loss, Discount

```
Profit% = (Profit/CP) x 100 | Loss% = (Loss/CP) x 100
SP = CP(1 + P%/100) or CP(1 - L%/100)
Successive discounts a%, b% -> Net = a+b-(ab/100)
Marked Price relation: SP = MP(1 - D%/100)
```

Q: A shopkeeper marks an item 40% above cost price and gives a 25% discount. Find his profit%.

A: Let CP=100. MP=140. SP=140×0.75=105. Profit% = **5%**.

Q: A man buys an article for Rs.450 and sells it for Rs.540. Find profit percentage.

A: Profit = 90. Profit% = (90/450)×100 = **20%**.

Q: The price of an item is reduced by 20% and then increased by 25%. What is the net change?

A: Net% = -20+25+(-20×25)/100 = 5 - 5 = **0% (no change)**.

Q: If the price of sugar falls by 10%, by what % should consumption increase to keep expenditure same?

A: Required increase = (10/(100-10))×100 = **11.11%**.

1.4 Simple & Compound Interest

```
SI = (P x R x T)/100
CI = P(1+R/100)^T - P
CI - SI (for 2 yrs) = P(R/100)^2
```

Q: Find SI on Rs.8000 at 9% p.a. for 3 years.

A: SI = (8000×9×3)/100 = **Rs.2160**.

Q: Find CI on Rs.10,000 at 10% p.a. for 2 years, compounded annually.

A: CI = 10000(1.1)^2 - 10000 = 12100-10000 = **Rs.2100**.

Q: The difference between CI and SI on a sum for 2 years at 5% is Rs.25. Find the sum.

A: P(R/100)^2 = 25 -> P(0.0025)=25 -> P = **Rs.10,000**.

1.5 Ratio, Proportion, Partnership, Mixtures & Allegation

```
Allegation: (Quantity of cheaper)/(Quantity of dearer) = (CP of dearer -
Mean)/(Mean - CP of cheaper)
Partnership profit ratio = (Capital x Time) ratio
```

Q: In what ratio must rice at Rs.60/kg be mixed with rice at Rs.75/kg so that the mixture costs Rs.65/kg?

A: By allegation: (75-65):(65-60) = 10:5 = **2:1**.

Q: A and B invest Rs.20,000 and Rs.30,000 for a business. Profit after 1 year is Rs.15,000. Find A's share.

A: Ratio = 20000:30000 = 2:3. A's share = (2/5)×15000 = **Rs.6000**.

Q: Divide Rs.1200 among A, B, C in the ratio 2:3:5.

A: Total parts=10. A=240, B=360, C=**600**.

1.6 Average, Ages, Time & Work, Pipes & Cisterns

```
Average = Sum of observations / Number of observations
Work: If A does work in 'a' days, A's 1-day work = 1/a
Combined: 1/a + 1/b = 1/(time together)
Pipes: Outlet pipe work is taken as negative
```

Q: The average of 5 numbers is 20. If one number is excluded, the average becomes 22. Find the excluded number.

A: Sum of 5 = 100. Sum of 4 = 88. Excluded number = **12**.

Q: A can do a work in 10 days, B in 15 days. In how many days will they finish it together?

A: 1/10+1/15 = 5/30 = 1/6. Together = **6 days**.

Q: A pipe fills a tank in 6 hours; another empties it in 8 hours. If both are opened, in how long will the tank fill?

A: 1/6 - 1/8 = 1/24. Tank fills in **24 hours**.

Q: The present age ratio of A and B is 4:5. After 6 years the ratio becomes 5:6. Find A's present age.

A: Let ages be 4x, 5x. (4x+6)/(5x+6)=5/6 -> 24x+36=25x+30 -> x=6. A's age = **24 years**.

1.7 Time, Speed, Distance, Trains, Boats & Streams

```
Speed = Distance/Time | km/hr to m/s: x(5/18) | m/s to km/hr: x(18/5)
Relative speed (same dir) = S1-S2 | (opp dir) = S1+S2
Downstream = B+S | Upstream = B-S | Boat speed=(D+U)/2, Stream=(D-U)/2
Train crossing pole: time=Length/Speed | crossing platform:
time=(L_train+L_platform)/Speed
```

Q: A train 150m long crosses a pole in 15 seconds. Find its speed in km/hr.

A: Speed = 150/15 = 10 m/s = 10×18/5 = **36 km/hr**.

Q: Two trains of length 120m and 100m run in opposite directions at 40km/hr and 32km/hr. Find time to cross.

A: Relative speed = 72km/hr = 20m/s. Total length = 220m. Time = 220/20 = **11 sec**.

Q: A boat's speed downstream is 15km/hr and upstream is 9km/hr. Find the boat and stream speed.

A: Boat = (15+9)/2 = 12km/hr. Stream = (15-9)/2 = **3km/hr**. Boat speed = **12km/hr**.

Q: A man covers a distance in 40 min at 30km/hr. What speed is needed to cover it in 30 min?

A: Distance = 30×(2/3) = 20km. Speed = 20/(0.5) = **40 km/hr**.

1.8 Clocks, Calendars, Permutation-Combination, Probability

```
Angle between hands = |11/2 M - 30H| degrees
Odd days: 1 ordinary yr=1, leap yr=2
nPr = n!/(n-r)! | nCr = n!/(r!(n-r)!)
Probability = Favourable outcomes / Total outcomes
```

Q: Find the angle between the hour and minute hand at 3:40.

A: |11/2(40) - 30(3)| = |220-90| = **130 degrees**.

Q: If 1st January 2024 was a Monday, what day was 1st January 2025? (2024 is leap year)

A: Leap year has 2 odd days, so add 2 to Monday = **Wednesday**.

Q: In how many ways can the letters of the word 'MANGO' be arranged?

A: All letters distinct, 5 letters -> **5! = 120** ways.

Q: A bag contains 4 red and 6 black balls. Find the probability of drawing 2 red balls.

A: P = C(4,2)/C(10,2) = 6/45 = **2/15**.

1.9 Geometry, Mensuration & Data Interpretation

```
Area: Rectangle=l×b | Triangle=1/2×b×h | Circle=πr^2
Volume: Cube=a^3 | Cuboid=lbh | Cylinder=πr^2h | Sphere=4/3πr^3
Curved Surface Area Cylinder=2πrh | Cone(slant l)=πrl
```

Q: Find the volume of a cylinder with radius 7cm and height 10cm. (Use pi=22/7)

A: V = 22/7 × 49 × 10 = **1540 cm³**.

Q: The area of a square is 144 sq.cm. Find its perimeter.

A: Side = 12cm. Perimeter = 4×12 = **48 cm**.

Q: In a pie chart, a sector representing 90 degrees corresponds to what percentage of the total?

A: Percentage = (90/360)×100 = **25%**.

Section 2

Logical Reasoning

2. Frequently Asked Questions — Logical Reasoning

2.1 Coding-Decoding & Number/Alphabet Series

Q: If CAT is coded as DBU, how is DOG coded?

A: Each letter +1: D->E, O->P, G->H = **EPH**.

Q: Find the next number: 2, 6, 12, 20, 30, ?

A: Differences: 4,6,8,10,12 -> next term = 30+12 = **42**.

Q: Find the odd one out: 3, 5, 11, 14, 17, 21

A: All except 14 are odd numbers; **14** is even, hence odd one out.

Q: Complete the series: Z, X, V, T, ?

A: Pattern: skip one letter backward (-2 each time) -> **R**.

2.2 Blood Relations & Direction Sense

Q: Pointing to a photograph, a man says, 'She is the daughter of my grandfather's only son.' How is she related to him?

A: Grandfather's only son = the man's father (or the man himself). If it's the man's father, she is his **sister**.

Q: A man walks 5km east, then 3km north, then 5km west. How far is he from the start and in which direction?

A: East and west cancel (5-5=0), only 3km north remains. Distance = **3km, North**.

2.3 Syllogism, Statement-Assumption, Statement-Conclusion

Q: Statements: All cats are dogs. All dogs are birds. Conclusion: All cats are birds.

A: By transitivity, this conclusion **follows**.

Q: Statement: 'Should smoking be banned in public places?' Assumption: Banning would reduce health hazards to non-smokers.

A: This assumption is implicit and **follows** since the question presumes a health-related motive.

2.4 Seating Arrangement & Puzzles

Q: Five friends A, B, C, D, E sit in a row. B sits immediately right of A. C sits at one end. D sits between B and E. If E sits second from left, find the arrangement.

A: Working through constraints: **C, E, D, A, B** is one valid order satisfying all clues (order may reverse depending on 'left/right' framing — always verify each clue against your final row).

2.5 Data Sufficiency & Input-Output

Q: Question: What is the area of the rectangle? Statement I: length is 10cm. Statement II: perimeter is 30cm.

A: From I alone, insufficient (need breadth). From II alone, insufficient. Combined: breadth = 15-10 = 5cm, area = 50 sq.cm, so **both statements together are sufficient, but neither alone is**.

Q: Input: 25 hat 36 cake 12 bat. Step 1 arranges words alphabetically, Step 2 arranges numbers ascending. What is the output after Step 2?

A: Step1: bat cake hat 25 36 12 (words alphabetically). Step2: numbers ascending -> **bat cake hat 12 25 36**.

2.6 Analogy, Classification, Ranking & Cubes/Dice

Q: Book : Author :: Painting : ?

A: A painting is created by a Painter, just as a book is created by an author -> **Painter**.

Q: Find the odd one: Apple, Mango, Potato, Banana

A: All are fruits except **Potato**, which is a vegetable.

Q: In a row of 30 students, Raj is 12th from the left. What is his position from the right?

A: Position from right = 30-12+1 = **19th**.

Q: A standard die shows 1 opposite 6, 2 opposite 5, 3 opposite 4. If '1' is on top and '2' faces you, which number is on the right face?

A: Using standard dice orientation rules, the right face is **3**.

Section 3

Verbal Ability

3. Frequently Asked Questions — Verbal Ability

3.1 Grammar Essentials

Q: Choose the correct sentence: (a) He don't like tea. (b) He doesn't like tea.

A: **(b)** is correct — third person singular takes 'doesn't', not 'don't'.

Q: Identify the error: 'Neither of the students have submitted their assignment.'

A: Error: 'have' should be **'has'** — 'neither' is singular and takes a singular verb.

Q: Convert to passive voice: 'The chef is cooking the meal.'

A: **'The meal is being cooked by the chef.'**

Q: Convert to indirect speech: She said, 'I am going to the market.'

A: **She said that she was going to the market.**

Q: Fill the blank with the correct preposition: 'She is good ___ mathematics.'

A: **at** — 'good at' is the correct collocation.

3.2 Vocabulary

Q: Find the synonym of 'Ephemeral'.

A: **Transient / short-lived**.

Q: Find the antonym of 'Frugal'.

A: **Extravagant / wasteful**.

Q: What does the idiom 'to bite the bullet' mean?

A: **To face a difficult situation with courage.**

Q: One word substitution: 'A person who can speak two languages.'

A: **Bilingual**.

Q: Choose the correctly spelled word: (a) Accomodate (b) Accommodate

A: **(b) Accommodate**.

3.3 Reading Comprehension, Para Jumbles & Cloze Test

Q: In a Para Jumble, how should you identify the opening sentence?

A: Look for the sentence that introduces the topic in general terms, without pronouns like 'this/it/he' referring to something not yet mentioned — that sentence is usually the **opener**.

Q: Cloze test strategy: 'The company reported a ___ increase in profits this quarter.' (a) marginal (b) marginalize (c) margin (d) marginally

A: The blank needs an adjective before 'increase' -> **(a) marginal**.

Section 4

Coding Fundamentals

4. Frequently Asked Questions — Coding Fundamentals (C / C++ / Java / Python)

4.1 Variables, Data Types & Operators

Q: What is the difference between '=' and '==' in programming?

A: '=' is the **assignment** operator (stores a value); '==' is the **equality comparison** operator (returns true/false).

Q: What is type casting? Give an example.

A: Converting one data type to another. Example in Python: **int('5') -> 5** converts string to integer.

```
int a = 10;
float b = 3.5;
int c = a + (int)b;  // c = 13 (explicit cast)
```

4.2 Loops & Functions

Q: What is the difference between 'while' and 'do-while' loops?

A: A 'while' loop checks the condition **before** executing the body; 'do-while' executes the body **at least once** before checking the condition.

```
# Python equivalent of do-while
while True:
    print('run once at least')
    if not condition:
        break
```

Q: What is a recursive function? Give a simple example.

A: A function that calls itself to solve smaller instances of the same problem. Example: factorial.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

4.3 Arrays & Strings

Q: How do you reverse a string in Python?

A: **s[::-1]** reverses string s using slicing.

Q: What is the time complexity of accessing an array element by index?

A: **O(1)** — constant time, since arrays support direct memory addressing.

Q: What is the difference between an array and a linked list?

A: Arrays have **contiguous memory** and O(1) indexed access but costly insertion/deletion; linked lists have **non-contiguous memory** with O(1) insertion/deletion at known nodes but O(n) access.

4.4 Pointers, OOP Basics & Exception Handling

Q: What is a pointer in C/C++?

A: A variable that **stores the memory address** of another variable, declared using '*' e.g. int *p = &a;

Q: What is the purpose of try-except (Python) or try-catch (Java/C++)?

A: To **gracefully handle runtime errors** without crashing the program, allowing fallback logic or clean error messages.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print('Cannot divide by zero')
```

Section 5

Data Structures

5. Frequently Asked Questions — Data Structures

5.1 Complexity Cheat Table

Data Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)*	O(1)*
Stack / Queue	O(n)	O(n)	O(1)	O(1)
Hash Table	-	O(1) avg	O(1) avg	O(1) avg
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)
Heap	-	O(n)	O(log n)	O(log n)

*O(1) at a known node/position; O(n) if search is required first.

5.2 Core Interview Q&A

Q: What is the difference between a Stack and a Queue?

A: Stack follows **LIFO** (Last In First Out); Queue follows **FIFO** (First In First Out).

Q: What is Hashing and why is it fast?

A: Hashing maps keys to array indices via a hash function, giving **average O(1)** lookup by avoiding linear search — collisions are handled via chaining or open addressing.

Q: What is the difference between a Binary Tree and a Binary Search Tree (BST)?

A: A Binary Tree has no ordering constraint; a **BST** requires left subtree values < node < right subtree values, enabling O(log n) search when balanced.

Q: What is a self-balancing tree, and why is AVL used?

A: A tree that automatically maintains O(log n) height via rotations after insert/delete. **AVL trees** keep the balance factor of every node within {-1,0,1}.

Q: Differentiate BFS and DFS traversal of a graph.

A: BFS explores **level by level** using a queue (shortest path in unweighted graphs); DFS explores **depth-first** using a stack/recursion (useful for cycle detection, topological sort).

Q: What is the use of a Trie data structure?

A: Efficient **prefix-based search** (autocomplete, dictionary lookup) in O(length of word) time.

Q: What is Union-Find (Disjoint Set Union) used for?

A: Efficiently tracking connected components/groups, used in **Kruskal's MST algorithm** and cycle detection, with near O(1) operations using path compression.

Section 6

Algorithms

6. Frequently Asked Questions — Algorithms

6.1 Searching & Sorting

Algorithm	Best	Average	Worst	Space
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

Q: Why is Binary Search O(log n)?

A: Each comparison eliminates **half** the search space, so the array size shrinks as n, n/2, n/4... reaching 1 in log2(n) steps.

Q: When would you prefer Merge Sort over Quick Sort?

A: When **stability** is required or worst-case guarantees matter (e.g., external sorting on large files) — Merge Sort is always O(n log n), while Quick Sort can degrade to O(n^2).

6.2 Sliding Window, Two Pointers & Prefix Sum

Q: What problem type does the Sliding Window technique solve efficiently?

A: Problems involving **contiguous subarrays/substrings** (max sum of size k, longest substring without repeats) — reduces O(n×k) brute force to O(n).

```
# Max sum subarray of size k
def max_sum(arr, k):
    window = sum(arr[:k])
    best = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i-k]
        best = max(best, window)
    return best
```

Q: What is the Two Pointer technique typically used for?

A: Problems on **sorted arrays** like pair-sum, removing duplicates, or merging — using two indices moving inward/forward to avoid nested loops.

6.3 Recursion, Backtracking, Greedy & DP

Q: What is the core difference between Greedy and Dynamic Programming?

A: Greedy makes the **locally optimal choice** at each step without reconsidering; DP explores all choices and **combines subproblem results**, guaranteeing global optimum when the problem has overlapping subproblems and optimal substructure.

Q: What is memoization?

A: Storing results of expensive function calls (usually recursive) so repeated calls with the same input return the **cached result** instead of recomputing — core to top-down DP.

Q: Give a real interview example of DP: 0/1 Knapsack recurrence.

A: dp[i][w] = max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i]) if wt[i] ≤ w, else dp[i-1][w].

Q: What is Backtracking? Give an example.

A: A recursive technique that builds a solution incrementally and **abandons ('backtracks')** a path as soon as it can't lead to a valid solution — e.g., N-Queens, Sudoku solver.

6.4 Graph Algorithms

Q: Name the algorithm used to find the shortest path in a weighted graph with non-negative edges.

A: **Dijkstra's Algorithm** (O((V+E) log V) with a min-heap).

Q: What algorithm finds Minimum Spanning Tree (MST)?

A: **Kruskal's** (sort edges + Union-Find) or **Prim's** (grow tree from a start node using a min-heap).

Section 7

Coding Practice Questions

7. Frequently Asked Coding Practice Questions (with Solutions)

7.1 Easy Level

Q: Check if a number is prime.

A: Check divisibility only up to sqrt(n).

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
# Time: O(sqrt n), Space: O(1)
```

Q: Reverse an array in place.

A: Use two-pointer swap from both ends.

```
def reverse_arr(a):
    l, r = 0, len(a)-1
    while l < r:
        a[l], a[r] = a[r], a[l]
        l += 1; r -= 1
    return a
# Time: O(n), Space: O(1)
```

Q: Check if a string is a palindrome.

A: Compare string with its reverse.

```
def is_palindrome(s):
    return s == s[::-1]
# Time: O(n), Space: O(n)
```

Q: Find the factorial of a number using recursion.

A: Base case n=0 returns 1; recursive multiply otherwise.

```
def fact(n):
    return 1 if n == 0 else n * fact(n-1)
```

```
# Time: O(n), Space: O(n) recursion stack
```

Q: Find the second largest element in an array.

A: Track largest and second-largest in a single pass.

```
def second_largest(a):
    first = second = float('-inf')
    for x in a:
        if x > first:
            second = first; first = x
        elif first > x > second:
            second = x
    return second
# Time: O(n), Space: O(1)
```

7.2 Medium Level

Q: Given an array, find two numbers that sum to a target (Two Sum).

A: Use a hash map to store complements seen so far — achieves O(n) instead of O(n^2) brute force.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
# Time: O(n), Space: O(n)
```

Q: Find the longest substring without repeating characters.

A: Use a sliding window with a set/dict tracking last seen index of each character.

```
def longest_unique(s):
    seen = {}
    start = best = 0
    for i, c in enumerate(s):
        if c in seen and seen[c] >= start:
            start = seen[c] + 1
        seen[c] = i
        best = max(best, i - start + 1)
    return best
# Time: O(n), Space: O(min(n, charset))
```

Q: Detect a cycle in a linked list.

A: Use Floyd's slow/fast pointer technique — if they meet, a cycle exists.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
# Time: O(n), Space: O(1)
```

Q: Find the level order traversal of a binary tree (BFS).

A: Use a queue, process node-by-node level by level.

```
from collections import deque
def level_order(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
# Time: O(n), Space: O(n)
```

7.3 Hard Level

Q: Find the length of the Longest Increasing Subsequence (LIS).

A: DP approach: dp[i] = length of LIS ending at i. O(n^2), or O(n log n) using binary search on a patience-sort array.

```
def lis(nums):
    if not nums: return 0
    dp = [1]*len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j]+1)
    return max(dp)
# Time: O(n^2), Space: O(n)
```

Q: Find the shortest path in a weighted graph (Dijkstra's Algorithm).

A: Use a min-heap; always expand the currently closest unvisited node, relax its neighbours.

```
import heapq
def dijkstra(graph, src):
    dist = {src: 0}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, float('inf')): continue
        for v, w in graph[u]:
            nd = d + w
            if nd < dist.get(v, float('inf')):
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist
# Time: O((V+E) log V), Space: O(V)
```

Section 8

DBMS • Operating Systems • Computer Networks

8.1 Frequently Asked Questions — DBMS

Q: What are the ACID properties of a transaction?

A: **Atomicity** (all-or-nothing), **Consistency** (valid state to valid state), **Isolation** (concurrent transactions don't interfere), **Durability** (committed changes persist even after failure).

Q: What is the difference between a Primary Key and a Foreign Key?

A: A **Primary Key** uniquely identifies each row in its own table (no NULLs, unique); a **Foreign Key** references a primary key in another table to enforce referential integrity.

Q: What is a Deadlock in DBMS, and how can it be prevented?

A: A situation where two or more transactions wait indefinitely for resources locked by each other. Prevented via **timeout, wait-die/wound-wait schemes, or resource ordering**.

Q: What is Functional Dependency?

A: A relationship where one attribute (or set) **uniquely determines** another, written as A -> B, meaning each value of A corresponds to exactly one value of B.

8.2 Frequently Asked Questions — Operating System

Q: What is the difference between a Process and a Thread?

A: A **Process** is an independent program in execution with its own memory space; a **Thread** is a lightweight unit of execution within a process that shares the process's memory space.

Q: What is a Deadlock, and what are its four necessary conditions?

A: A state where processes wait forever for resources held by each other. Conditions: **Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait** — all four must hold simultaneously.

Q: Differentiate Paging and Segmentation.

A: **Paging** divides memory into fixed-size blocks (pages) to avoid external fragmentation; **Segmentation** divides memory into variable-sized logical segments (code, stack, data) matching program structure, but can suffer external fragmentation.

Q: What is the difference between a Mutex and a Semaphore?

A: A **Mutex** is a locking mechanism allowing only one thread to access a resource at a time (binary, ownership-based); a **Semaphore** is a signaling counter that can allow multiple threads up to a limit (counting) and has no ownership concept.

Q: What is Virtual Memory?

A: A memory management technique that uses disk space to **simulate additional RAM**, allowing programs larger than physical memory to run, via paging and swapping.

8.3 Frequently Asked Questions — Computer Networks

Q: What are the 7 layers of the OSI model?

A: **Physical, Data Link, Network, Transport, Session, Presentation, Application** (mnemonic: Please Do Not Throw Sausage Pizza Away).

Q: What is the difference between TCP and UDP?

A: **TCP** is connection-oriented, reliable, ordered (uses handshake, acknowledgements); **UDP** is connectionless, faster, no delivery guarantee — used for streaming/gaming where speed matters more than reliability.

Q: What happens when you type a URL into a browser? (Common TCS/HR-technical hybrid question)

A: Briefly: **DNS resolution** (domain -> IP) → **TCP handshake** with server → **HTTPS/TLS negotiation** → browser sends **HTTP GET request** → server responds with HTML/assets → browser renders the page.

Q: What is the difference between HTTP and HTTPS?

A: **HTTPS** is HTTP layered over **TLS/SSL encryption**, securing data in transit; plain HTTP sends data unencrypted.

Q: What is DNS and why is it needed?

A: The **Domain Name System** translates human-readable domain names (e.g., google.com) into IP addresses that computers use to route traffic.

Section 9

Object-Oriented Programming

9. Frequently Asked Questions — OOP Concepts (Python & Java)

Q: Define the four pillars of OOP.

A: **Encapsulation** (bundling data+methods, restricting direct access), **Inheritance** (reusing a parent class's properties), **Polymorphism** (same interface, different implementations), **Abstraction** (hiding implementation details, exposing only essential features).

```
# Python example
class Animal:
    def speak(self):
        raise NotImplementedError

class Dog(Animal):          # Inheritance
    def speak(self):        # Polymorphism (overriding)
        return 'Woof'

class Cat(Animal):
    def speak(self):
        return 'Meow'
```

```
// Java example
abstract class Animal {
    abstract String speak();
}
class Dog extends Animal {
    String speak() { return "Woof"; }
}
```

Q: What is the difference between Method Overloading and Method Overriding?

A: **Overloading**: same method name, different parameters, resolved at **compile time** (same class). **Overriding**: subclass redefines a parent method with the same signature, resolved at **runtime** (polymorphism across classes).

Q: What is an Abstract Class vs an Interface?

A: An **Abstract Class** can have both implemented and unimplemented methods, and supports single inheritance; an **Interface** traditionally declares only method signatures (contracts) and a class can

implement multiple interfaces.

Section 10

HR Interview

10. Frequently Asked HR Interview Questions

Q: Tell me about yourself.

A: Structure it as: **Present** (who you are academically/professionally) → **Past** (relevant projects/experience) → **Future** (why this role fits your goals). Example: 'I'm a final-year M.Sc. Data Science student at SASTRA University with a strong academic record. I've worked on projects involving Python, SQL, and Power BI for data analysis, and I'm now looking to apply these skills in a data-driven role at TCS, where I can contribute to real business problems while growing technically.'

Q: What are your strengths?

A: Pick 2-3 **genuine, role-relevant** strengths and back each with a one-line example. E.g., 'Analytical thinking — in my thesis project I identified a data quality issue that changed our entire modeling approach.'

Q: What is your biggest weakness?

A: Choose a real but **non-critical** weakness and show how you're actively improving it. E.g., 'I used to spend too long perfecting small details; I've started timeboxing tasks to balance quality with deadlines.'

Q: Why TCS?

A: Mention TCS's **scale, learning culture, and project diversity**. E.g., 'TCS gives freshers exposure to diverse domains and clients globally, along with strong learning platforms like TCS iON — that combination of scale and structured growth is exactly what I'm looking for early in my career.'

Q: Why should we hire you?

A: Connect your **specific skills** to the role's needs, and show enthusiasm. E.g., 'My data science coursework and hands-on project experience with SQL and Python align directly with what data-driven roles need, and I'm eager to learn and contribute from day one.'

Q: Tell me about a time you faced failure. How did you handle it?

A: Use the **STAR method** (Situation, Task, Action, Result) and end with what you learned, not the failure itself.

Q: Where do you see yourself in 5 years?

A: Show **ambition aligned with the company**, not a generic answer. E.g., 'I see myself growing into a senior data analyst or data scientist role, having built strong domain expertise while contributing to impactful projects within TCS.'

Section 11

30-Day Study Plan

11. 30-Day Study Plan

This plan assumes ~3-4 focused hours/day. Adjust pace based on your baseline strength in each area.

Day(s)	Focus	Daily Tasks
1-2	Number System, HCF/LCM, Percentages	Theory + formulas + 20 Q&A each; revise divisibility tricks.
3-4	Profit/Loss, SI/CI, Ratio & Partnership	Solve mixed problems; time yourself (1.5 min/question).
5-6	Time-Work, Pipes, TSD, Trains, Boats	Focus on relative speed & work-rate shortcuts.
7	Full Quant Mock + Review	Attempt a timed set; log error patterns, not just scores.
8-9	Coding-Decoding, Series, Blood Relations, Direction	Practice diagram-based solving daily.
10-11	Syllogism, Seating Arrangement, Puzzles	Do at least 5 puzzles with grid/diagram method.
12	Full Reasoning Mock + Review	Time-box each question type separately to find weak spots.
13-14	Grammar (tenses, SVA, voice, speech)	Do error-spotting drills; revise 30 new vocab words/day.
15-16	Vocabulary, RC, Para Jumbles, Cloze Test	Read 1 editorial/day; practice 2 RC passages daily.
17	Full Verbal Mock + Review	Track time spent per RC passage; aim under 6 min each.
18-19	Coding Basics (loops, functions, arrays, strings)	Solve 8-10 easy problems in your chosen language.
20-21	Data Structures (arrays, LL, stack, queue, hashing, trees)	Implement each structure from scratch once.
22-24	Algorithms (sorting, searching, sliding window, DP basics)	Solve 2 medium DP/graph problems daily.
25	DBMS + OS core concepts	Revise ACID, normalization, deadlock, paging via the FAQs in this book.
26	Computer Networks + OOP	Revise OSI layers, TCP/UDP, and the 4 OOP pillars with code examples.
27	Resume-based mock technical interview	Rehearse explaining every project out loud, timed to 2 min each.
28	HR round preparation	Draft & rehearse answers to all the HR FAQ questions in this book.
29	Full-length mixed mock test	Simulate exact exam timing; review every wrong answer's root cause.
30	Final revision day	Skim through all FAQ sections once; light review, get good rest.